

Лабораторна робота. Варіант 4: швидкодія std::count_if

Виконав: Дмитро Демчук, група К-25

1. Методика

- **Збірка.** GCC 16 (g++-16), C++20; політики std::execution працюють поверх oneTBB (Apple Clang їх не підтримує). make run збирає програму з -O0 та -O3 і записує вивід у results/. Тестова машина: hardware_concurrency = 8.
- **Дані.** Цілі числа з [2, 100000] (mt19937, seed 42), три обсяги N = 100 000; 1 000 000; 10 000 000. Файли data/seq_N.txt: перший рядок - N, другий - N чисел через пробіл. Усі режими й обидві збірки читають однакові файли; якщо їх немає, програма створює їх сама, параметр --new створює нові.
- **Предикати.** isEvenNumber ($x \% 2 == 0$) - дешевий; isPrime (пробні ділення до $\sqrt{x}$) - дорогий.
- **Таймери.** total - читання файлу, count_if і вивід результату; compute - лише count_if на даних у RAM.
- **Власний алгоритм.** Масив ділиться на K частин; кожну рахує окремий std::thread послідовним count_if; часткові лічильники складає std::reduce.

2. Бібліотечний count_if: без політики та з політиками

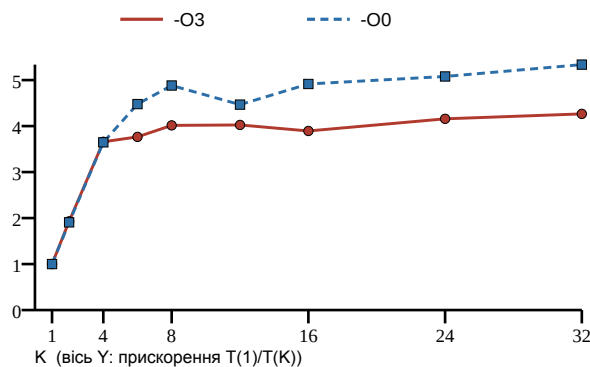
N	предикат	none	seq	par	par_unseq
100 000	even	0.094	0.094	0.038	0.029
100 000	prime	2.668	2.678	0.611	0.837
1 000 000	even	0.938	0.937	0.253	0.171
1 000 000	prime	26.983	27.398	6.950	6.630
10 000 000	even	9.977	9.639	1.568	1.839
10 000 000	prime	267.834	268.373	60.591	61.518

Таблиця 1. Час compute, мс (-O3). Результат count_if однаковий у всіх режимах.

На дорогому предикаті par/par_unseq прискорюють count_if у 4.4 раза (268 → 61 мс при N = 10 000 000): роботи на елемент багато, накладні витрати розподілу малі. На дешевому предикаті виграш є (6.1 раза), але менший і нестабільніший, бо самого обчислення мало; на малих N час (соті частки мс) порівнянний із шумом. «Без політики» та seq однопотоківі й відрізняються лише в межах похибки. Таймер total для even (313 мс проти compute 10.0 мс) майже весь складається з читання текстового файлу, тому для порівняння політик використано compute.

3. Власний паралельний алгоритм (isPrime, N = 10 000 000)

K	-O3, мс	прискор.	-O0, мс	прискор.
1	265.7	1.00	714.2	1.00
2	137.7	1.93	374.5	1.91
4	72.6	3.66	195.9	3.65
6	70.6	3.76	159.5	4.48
8	66.2	4.02	146.3	4.88
12	66.0	4.02	159.9	4.47
16	68.2	3.89	145.3	4.92
24	63.9	4.16	140.6	5.08
32	62.3	4.26	133.8	5.34



Таблиця 2 і рисунок. Залежність часу від K.

Найкраще K. Формальний мінімум: K = 32 (62.3 мс) на -O3 і K = 32 (133.8 мс) на -O0, тобто 4 і 4 × hardware_concurrency (8). Але вже при K = 8 час лише на 6% і 9% більший, а при K ≥ 8 він коливається в межах 62-68 мс (-O3) і 134-160 мс (-O0): положення мінімуму всередині цього плато випадкове. Практично оптимальне K близьке до кількості апаратних потоків (8); більші K виграшу не дають.

Закон зміни часу. При малих K час падає приблизно як $T(1)/K$ (прискорення 1.93 при K = 2 і 3.66 при K = 4), далі виграш слабшає і на плато прискорення становить 4.0-4.3, тобто менше за 8 (ймовірно, ядра нерівноцінні, а розбиття додає накладні витрати). Подальше збільшення K час майже не змінює: вартість створення потоків мала порівняно з роботою частин.

4. Вплив рівня оптимізації

режим (isPrime, N = 10 000 000)	-O0, мс	-O3, мс	прискорення
none	715.2	267.8	2.67 ×
seq	713.6	268.4	2.66 ×
par	139.1	60.6	2.30 ×
par_unseq	136.9	61.5	2.23 ×
власний, K = 8	146.3	66.2	2.21 ×

Таблиця 3. Час compute залежно від рівня оптимізації.

Лише рівень оптимізації прискорює код у 2.2-2.7 раза без жодних змін і потоків. Тому політики виконання порівнювалися в межах одного рівня оптимізації: інакше «виграш від паралелізму» частково був би виграшем від оптимізації.

5. Висновки

- Політики par/par_unseq суттєво прискорюють count_if лише для важкого предиката (4.4 раза проти 6.1 раза для дешевого); «без політики» і seq еквівалентні.
- Власний алгоритм поводить ся як par: $T(K) \approx T(1)/K$ при малих K, далі плато; практично найкраще $K \approx \text{hardware_concurrency}$ (8), формальне K з мінімальним часом усередині плато нестабільне.
- Оптимізація -O3 дає 2.2-2.7 раза - співмірно з паралелізмом, тому порівнювати треба в межах одного рівня оптимізації.